Pont AI · SaaS Technical Implementation Plan

> Focus: multi-tenancy for **Xia** (the core engine), SaaS operating model, elastic scaling strategy, and account provisioning
> Last updated: 2026-04-18
> Version: **v2.0 (Cloud-Native Elastic Architecture)**
> Companion docs: `[引擎一+定制化-商业化落地方案.md](./引擎一+定制化-商业化落地方案.md)` · `[引擎一-订阅与用量收入规划.md](./引擎一-订阅与用量收入规划.md)`
> Audience: technical co-founder · engineers · operations

---

0. Core thesis (read this first)

0.1 Technical path

Xia (「虾」) = technical foundation (no rewrite; evolutionary upgrade). SaaS = outer shell (new). Custom delivery = specialization (shared foundation).

```
┌──────────────────────────────────┐
│    SaaS shell (build in 1–2 mo.)  │
│  signup / subscription / console  │
│  quotas / billing                  │
└──────────────┬────────────────────┘
               │
┌──────────────▼────────────────────┐
│    Xia engine (core; continuous)  │
│  crawl / analyze / generate /      │
│  publish / monitor                 │
└──────────────┬────────────────────┘
               │
┌──────────────▼────────────────────┐
│  Custom delivery (per customer;   │
│  reuses foundation)               │
│  industry templates / on-prem pkg  │
│  integration layer                 │
└──────────────────────────────────┘
```

0.2 Design principles (backbone of this document)

This version is not "pick one stack and fight it forever," but a flexible architecture around **four principles**:

Principle	Meaning
P1 · Deployment abstraction	The same codebase deploys to Docker Compose / K3s / K8s / Serverless—any of them. Technology choices are **swappable modules**, not irreversible decisions
P2 · Event-driven + elastic on demand	Resources are not "reserved"; they **start on events and return when done**. No vertical scaling by "adding more VPS"
P3 · Multi-tenant security isolation	Isolate data, compute, and network—but tiers can graduate (shared → soft isolation → hard isolation)
P4 · Progressive evolution	Each phase's architecture is a **subset of the next**—no destructive rewrites

0.3 Most urgent technical problem

> **"1 VPS per Xia instance" does not work at a 999/month price point. "Add VPS" is not the answer either.**

Rough math:

- One basic VPS (2C4G, domestic China): **150–300 RMB/month**
- LLM API + proxy + other: **200–400 RMB/month**
- Infra cost per customer: **~400–700 RMB/month**
- Gross margin only **30%–60%**

Traditional scaling (wrong): add more VPS. Why it fails:

- Traffic peaks and troughs (support active by day; content jobs often at night)—always-on VPS sits idle ~70% of the time
- No coordinator for how many jobs run per VPS—ops is manual

- If a host dies, migration is manual
- Cost does not drop linearly with VPS count; coordination overhead rises

Modern scaling (right):

- **Base pool** (fixed, 2–3 nodes, SLA)
- **Elastic pool** (spin up on demand when work arrives)
- **Serverless burst pool** (overflow to vendor serverless containers at peak, billed per second)

Goal: infra cost per customer from 400 → **30–80 RMB/month** (usage-based), gross margin **92%+**.

0.4 MVP loop to prove in 1–2 months

```
Customer signup → choose plan → pay (3-month minimum)
    → auto provisioning (not "spin a VPS"—write tenant row + allocate quota)
    → task arrives → control plane schedules workers → execute → bill
    → customer console shows results → renew at expiry
```

Seven self-service steps, no manual ops = SaaS is real.

---

1. Assessment of Xia today

1.1 Current state

Area	Today	Issue
Operation	1 VPS per customer, isolated Xia deploy	High cost, heavy ops, poor scale
Deployment	Manual / semi-auto	New customers need human touch
Isolation	VPS-level (physical)	Wastes capacity; one customer on 2C4G often uses only 10%–30%
Monitoring	Single-machine logs	Debug via SSH
Accounts	Per-customer cookies/sessions	No unified identity pool
Quotas	None	Customers run unlimited workloads
Billing	None (just VPS rent)	No usage-based pricing
Delivery quality	Content, placement, monitoring iterating	Optimize while running

1.2 Core decision: no rewrite; evolve

Do not:

- ❌ Rewrite the Xia core (3–6 months; interrupts the business)
- ❌ Wait until Xia is "perfect" before SaaS (never happens)
- ❌ One VPS per customer until 50 customers (cost will crush you)

Do:

- ✅ Change Xia from "**long-running process**" to "**event-driven task worker**"
- ✅ Add a **control plane** for scheduling, billing, monitoring
- ✅ Introduce a **modern orchestration layer** (K3s/K8s/Serverless), schedule resources on demand
- ✅ Iterate the **Xia core in parallel**—each upgrade changes image/function version, not architecture

---

2. Target architecture: from "layers" to "abstractions"

2.1 Overall architecture (four abstractions)

```
┌──────────────────────────────────────────────────────────┐
│  ① User plane                                            │
│     pont-ai.com   +   app.pont-ai.com                    │
│     (marketing+SEO)  (signup+console+payment)            │
└──────────────────────────┬───────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────┐
│  ② Control plane — stateless, HA                         │
│     tenant · subscription/billing · orchestration ·      │
│     quotas · monitoring · account pool                    │
└──────────────────────────┬───────────────────────────────┘
                           │  events/tasks (Redis Streams / Kafka / NATS)
┌──────────────────────────▼───────────────────────────────┐
│  ③ Deployment abstraction — swappable orchestration      │
│     K3s (MVP) → K8s (growth) → Serverless (burst)         │
│     + KEDA (event-driven autoscale)                       │
│     + Temporal (workflow engine for long jobs)            │
└──────────────────────────┬───────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────┐
│  ④ Data plane — stateful, tenant-isolated                 │
│     Xia worker Pods × N (start/stop on demand)            │
│     PostgreSQL + pgvector · Redis · S3                    │
│     proxy pool + account pool + LLM routing               │
└──────────────────────────────────────────────────────────┘
```

2.2 Key abstraction: deployment layer (source of flexibility)

This is the biggest change vs. the prior version. We do not hard-code "how it runs"; we abstract it:

```
Control plane calls a single API:
    dispatch(task, tenant_id)
    kill(task_id)
    scale(target_count)

Underneath can be:
    - Phase 1: K3s + Docker pods
    - Phase 2: K8s + KEDA autoscale
    - Phase 3: Alibaba SAE / Knative serverless containers
    - Custom customer: Docker Compose single host
    - Overseas: AWS EKS / GCP GKE

Same code; deployment can switch across clouds and architectures.
```

How to implement?

Use Kubernetes APIs or **Dapr** (Distributed Application Runtime, Microsoft OSS). Dapr abstracts "invoke tasks, state, pub/sub" behind one interface—you run on K8s or a single local machine **without code changes**.

> This design solves two things:
>
> 1. MVP can start on minimal K3s (even Docker Compose)
> 2. Large on-prem deals overseas: customer runs `helm install` and goes

2.3 Technology matrix (not fixed—choose per scenario)

Orchestration

Phase	Recommended	Alternatives	When to switch
MVP (0–30 customers)	**K3s** (lightweight K8s, single master)	Docker Swarm / Nomad	Default for MVP; one machine can run it
Growth (30–300)	**K3s multi-node + KEDA**	Alibaba ACK / Tencent TKE	More customers → HA
Scale (300+)	**Managed K8s + serverless overflow**	Self-managed K8s	Full cloud-native elasticity

Phase	Recommended	Alternatives	When to switch
On-prem	Docker Compose + Helm chart	Single-node K3s	Overseas / strong compliance

Recommended start: K3s. Reasons:

- Single binary, ~5 minutes to stand up
- Full K8s API—smooth upgrade path later
- Small footprint—runs on 2C4G
- Fits small teams

Task / workflow engine

Option	Fit	Pros / cons
Redis Streams + BullMQ	Simple queues	Pro: light; con: complex workflows are hard
Temporal	**Complex workflows, long tasks, retries**	**Pro: durable state, observability, fits Xia**; con: learning curve
Kafka	Event streaming, high throughput	Pro: standard; con: heavy ops
NATS JetStream	Lightweight streaming	Pro: simple; con: smaller ecosystem

Recommendation: Temporal. Reasons:

- Xia's jobs are **long, multi-step, stateful** (keywords → generate → review → publish → monitor)—Temporal's strength
- Automatic retries; visualize each step
- MVP can start with Redis Streams; migrate to Temporal by M3

Elastic autoscaling

Tech	Role
HPA (native K8s)	Scale on CPU/memory
KEDA	**Scale on event sources (queue depth, MQ depth, Redis list length)**—built for SaaS
Cluster Autoscaler	Node-level scale (add cloud VMs when nodes are insufficient)
Vendor serverless containers	Alibaba SAE / Tencent serverless containers—per-second billing, second-scale cold start

Recommended combo:

- **Steady traffic**: K3s + KEDA (scale worker pods on queue depth)
- **Bursts**: overflow to vendor serverless containers (SAE / SCF)
- **Nightly batch**: scale out ~10× at night, scale back in the morning

Multi-tenant isolation

Tier	Approach	Fit
Data (most important)	PostgreSQL RLS + mandatory `tenant_id`	All customers
Compute (soft)	K8s Namespace + ResourceQuota	Most SaaS customers
Compute (hard)	Dedicated node / cluster	Custom ¥5M deals
Network	NetworkPolicy + Istio	Strong compliance
Extreme	Firecracker microVMs / gVisor	On-prem deployments

Emerging option (consider M9+): WebAssembly (Wasm) + WASI

- Each tenant in an isolated Wasm sandbox
- Millisecond cold start—~100× faster than containers
- ~1/10 memory vs. containers
- Projects: Fermyon Spin, wasmCloud

- Ecosystem still maturing—but **most promising multi-tenant isolation for 2026–2027**, worth tracking

Observability

Tech	Role
OpenTelemetry	Standard tracing/metrics/logs
Prometheus + Grafana	Metrics + dashboards
Loki	Log aggregation (lightweight)
Tempo	Distributed tracing
Sentry	Error tracking

Recommendation: OpenTelemetry + Grafana LGTM stack (Loki + Grafana + Tempo + Mimir). All OSS; one stack for logs/traces/metrics.

Data layer

Component	Choice	Rationale
Primary DB	**PostgreSQL 16 + pgvector**	Transactions + vectors in one place
Cache	**Redis 7**	Cache + queue
Object storage	Alibaba OSS / Tencent COS	Cheap, stable
Time series (metrics)	**VictoriaMetrics**	High-performance Prometheus alternative
Full-text search (optional)	**Meilisearch / Typesense**	Lighter than Elasticsearch

2.4 Selection summary

Layer	MVP (M0–M2)	Growth (M3–M6)	Scale (M6+)
Orchestration	Single-node K3s	Multi-node K3s + KEDA	Managed K8s + serverless
Tasks	Redis Streams + BullMQ	Temporal	Temporal + Kafka
Isolation	RLS + Namespace	+ ResourceQuota	+ NetworkPolicy
Observability	Grafana Cloud (free tier)	Self-hosted LGTM	+ OpenTelemetry
Abstraction	—	Dapr (optional)	Dapr + service mesh

---

3. Xia evolution: from containers to event-driven

This is the **technical core** of the document—three phases; each phase is a shippable product.

3.1 Evolution goals

```
Phase 1: Task-ify        → Change how Xia runs (M0–M3)
          ↓
Phase 2: Orchestrate     → Add elastic scheduling (M3–M6)
          ↓
Phase 3: Event-driven /  → Move toward serverless (M6+)
          function-ify
```

Xia core code barely moves—**only how it is scheduled changes**.

3.2 Phase 1 (M0–M3) · Task-ify

Goal: Change Xia from "**one long-running process per VPS**" to "**worker pulling from a queue**".

Changes

1. **Split Xia into stateless workers**

- Input: `{task_type, tenant_id, payload, config}`
- Output: `{result, metrics, status}`
- On start: pull task → execute → report → repeat

2. **Package workers as images** (Docker for packaging—not "we only deploy with Docker")

- `pont/xia-worker:v1`
- Runs on K3s, Docker Compose, serverless containers—anywhere
- Image does not bind to a single deployment style

3. **Control plane enqueues work**

```
Customer triggers a job (e.g. "generate 50 articles")
   ↓
Control plane splits into 50 subtasks in the queue
   ↓
Worker pool pulls concurrently
   ↓
Each completion: bill + persist customer data
```

4. **Baseline isolation**

- Each worker pod capped at 1C1G
- K3s Namespace isolation
- DB RLS for data boundaries

Phase 1 cost

Item	Unit cost
K3s: 1 master + 2 worker nodes (8C16G × 2)	~1,800 RMB/month
Customers supported	**20–30 entry / 10–15 pro**
Infra per entry customer	**~60–90 RMB/month**
Gross margin on entry 999 plan	**~90%+**

Phase 1 wins:

- Xia core untouched; short retrofit window
- One K3s investment carries to next phase (K3s → K8s smoothly)
- ~80% cost reduction

3.3 Phase 2 (M3–M6) · Orchestration + elasticity

Goal: System **grows and shrinks automatically**.

Changes

1. **Add KEDA**

- KEDA watches queue depth
- Backlog → more worker pods
- Idle → scale to minimum (even zero)

- **Example config**:

```
# illustrative
ScaledObject:
  minReplicas: 2        # keep at least 2 for responsiveness
  maxReplicas: 50       # up to 50 at peak
  triggers:
    - type: redis-streams
      metadata:
        consumerGroup: xia-workers
        streamLength: "10"   # one worker per 10 tasks
```

2. **Upgrade task engine to Temporal**

- Model Xia flows (keywords → generate → review → publish) as Temporal workflows
- Retries, traceability, durable state
- Complex custom flows (e.g. ERP → orders → CS scripts) become tractable

3. **Overflow to serverless**

- Base K3s for steady load
- Peaks overflow to Alibaba SAE or Tencent elastic containers
- **Per-second billing**—pay only during bursts

4. **Abstract deployment (optional Dapr)**

- Dapr sidecar
- Xia talks to state/pub-sub/service invoke via Dapr
- Swap infra underneath freely

Phase 2 cost

Item	Unit cost
K3s base cluster (4 nodes)	3,500 RMB/month
DB + middleware	2,500 RMB/month
Serverless overflow (avg)	1,000 RMB/month
Customers	**80–150 entry / 30–60 pro**
Infra per entry customer	**~25–40 RMB/month**

3.4 Phase 3 (M6+) · Event-driven / function-ify

Goal: Full cloud-native; Xia is not a “process” but a “function”.

Architecture

```
Event in (payment, cron, webhook)
    ↓
Function platform starts xia-function
    ↓
Release resources right after execution
    ↓
Write results, bill
```

Options

Option	Fit
Knative	Native serverless on K8s
Alibaba FC / Tencent SCF	Domestic FaaS, pay per invocation
OpenFaaS	Self-hosted functions
Fermyon Spin (Wasm)	Forward-looking; ms cold start

Option	Fit

Why go here

- After 500+ customers, fixed-node economics weaken
- Usage is highly uneven (some customers 1,000 runs/day, others 10/month)—per-invocation is fairer
- On-prem customers can ship Knative + `xia-function` in one package—**one-click deploy**

Phase 3 cost

Customer scale	Monthly infra	Per-customer cost
500	30k–50k RMB	**~60–100 RMB**
1000	60k–80k RMB	**~60–80 RMB**
3000	150k–200k RMB	**~50–70 RMB**

Larger scale → lower per-customer cost—modern SaaS leverage.

---

4. SaaS operating model (end-to-end)

4.1 Full journey

```
Customer sees "30-day free trial" on pont-ai.com
  → app.pont-ai.com/signup
  → 3-step signup (phone/email/org) + industry
  → auto trial tenant
    • control plane creates tenant_id
    • assigns quota
    • initializes knowledge base (empty)
  → 4-step onboarding (plan intent / upload / platform auth / keywords)
  → data ingested to KB (pgvector chunk + embed)
  → first batch within 5 minutes (expectation design)
  → console shows progress + preview
  → pay within 30 days → trial converts → quota upgrade
  → daily operation:
    • cron schedules tasks
    • KEDA scales workers on queue depth
    • each task records usage
    • overage auto-charges
  → renewal reminder 7 days before expiry
  → no renew → 7-day read-only grace → suspend, retain 90 days → delete
```

4.2 Control plane responsibilities

Service	Responsibility	SLA
Tenant	Accounts/users/permissions/plans	99.9%
Subscription/billing	Orders/payment/invoices/renewal/up-downgrade	99.9%
Task orchestration	Split tasks, enqueue, retry	99.5%
Quotas	Debit, overage alerts	99.9%
Account pool	Platform account allocation, rotation	99%
Proxy pool	IP proxy management	99%
Monitoring/alerts	System health + business metrics	—

4.3 Core data model

```
-- Tenants
tenants (id, company_name, plan_id, status, trial_ends_at, created_at)

-- Subscriptions
subscriptions (id, tenant_id, plan_id, period, start_at, end_at, amount, status)

-- Orders
orders (id, tenant_id, amount, payment_method, paid_at, invoice_id)

-- Quotas
quotas (tenant_id, resource_type, period, total, used, overage_billed)

-- Tasks
tasks (id, tenant_id, type, status, payload, result, created_at, completed_at)

-- Worker heartbeats
workers (id, pod_name, status, current_task_id, heartbeat_at)

-- Account pool
accounts (id, platform, status, assigned_at, last_used_at)

-- Proxy pool
proxies (id, type, region, status, bandwidth_used)

-- All business tables must include tenant_id and Row Level Security
```

---

5. Provisioning and subscription policy

5.1 Minimum commitment

Plan	Monthly	Quarterly (minimum)	Annual (recommended)
Entry 999	▯ not offered	**3 months 2,997** (standard)	9,990 (save 2 months)
Pro 2,999	3,599 (+20% premium)	6 months 17,994 (standard)	29,990 (save 2 months)
Flagship 9,999	▯ not offered	6 months 59,994	99,990 (recommended)

Intent:

1. Entry: no monthly—avoid “try and churn”
2. Pro monthly +20%—steer to quarterly/annual
3. Flagship: semi-annual/annual only—¥9,999 deals fit annual contracts

5.2 Trial policy

Item	Design
Trial length	30 days
Trial resources	**20%** of paid
Trial limits	1 platform, 1 language, no long history
Activation	**Real company name + business type**
After expiry	Data read-only 30 days

5.3 Anti-abuse

- No duplicate trials for same phone/email/company
- Max 1 signup per IP per 24h
- Freeze after 30 days unpaid
- Rules engine (simple: Redis + Lua)

5.4 Overage pricing

Resource	Included	Overage unit
Content gen	Entry 100 / Pro 500 / Flagship unlimited	3 RMB/article
CS chat	Entry 500 / Pro 3000 / Flagship unlimited	1 RMB/session
GEO monitoring	Entry 5 / Pro 50 / Flagship unlimited	10 RMB/keyword/month
Platform connections	Entry 2 / Pro 10 / Flagship unlimited	99 RMB/platform/month
KB documents	Entry 10 / Pro 100 / Flagship unlimited	10 RMB/doc

Mechanics:

- Soft cap: no hard stop + auto charge + SMS
- Hard cap: balance zero and no top-up → suspend after 3-day grace
- At ~60% overage, upsell (modal + in-app message)

5.5 Account state machine

```
[Trial] ──pay──▶ [Active]
    └──no pay in 30d──▶ [Frozen] ──60d──▶ [Deleted]

[Active] ──expired──▶ [7d read-only grace] ──▶ [Frozen] ──90d──▶ [Deleted]
     ──arrears──▶ [3d past due] ──▶ [Frozen]
     ──cancel──▶ [effective end of term]

[Frozen] ──top-up/renew──▶ [Active]
```

---

6. Infrastructure and cost model

6.1 First clarify: the software is free

People hear K3s, KEDA, Kubernetes and assume huge license cost. **Core components here are OSS—no commercial license fees.**

Component	Cost	License
K3s	**Free**	Apache 2.0
KEDA	**Free**	Apache 2.0, CNCF
Kubernetes	**Free**	Apache 2.0
Docker (engine)	**Free**	Apache 2.0
PostgreSQL + pgvector	**Free**	PostgreSQL License
Redis 7.x	**Free** (community enough)	RSAL
Temporal (OSS)	**Free**	MIT
Grafana + Prometheus + Loki	**Free**	AGPL / Apache 2.0
OpenTelemetry	**Free**	Apache 2.0

Component	Cost	License
Dify (optional)	Free	Apache 2.0

You actually pay for four buckets:

1. **Cloud compute** (VPS / ECS)
2. **Managed DB & storage** (RDS, managed Redis, OSS)
3. **LLM API calls**
4. **Third-party** (SMS, proxies, payment fees, captchas)

Below: realistic bills by phase (Alibaba Cloud 2025–2026 pricing; Tencent similar).

6.2 Where the money goes

```
Monthly total =
   cloud compute (40–50%)
 + DB/cache (15–20%)
 + LLM API (20–30%)  ← biggest variable
 + storage/CDN/traffic (5–10%)
 + third-party (5–10%)
 + proxy/account pool (0–15%; MVP can be 0)
```

6.3 MVP detailed bill (M1–M3, <30 customers)

MVP **does not need everything on day one**—scale with customer count. Understand **control** vs **worker** nodes:

Node	Runs	Load profile
Control	Tenant mgmt, payment, scheduling, API, frontend	Light, low traffic
Worker	Xia worker pods, job execution	Heavy, grows with customers

Three MVP tiers (pick by customer count)

Tier	Hardware	Monthly	Fit	Tradeoff
A · Minimal	Single 16C32G (control+worker)	**1,600 RMB**	<10 customers (M1–M2)	No HA; reboot on failure (~10 min)
B · Economical	4C8G control + one 8C16G worker	**1,200 RMB**	10–20 customers	Same as above
C · Standard	4C8G control + two 8C16G workers	**2,000 RMB**	20–30 + need HA	Highest cost

Suggested path: A (M1–M2) → B (M3, upgrade worker) → C (after M4 toward growth architecture).

Tier A full bill

Item	Spec	Monthly
All-in-one VPS	Alibaba 16C32G	1,600 RMB
PostgreSQL RDS	2C4G + 200GB	500 RMB
Redis	1C2G	200 RMB
OSS + CDN	usage-based	300 RMB
Domain + SSL	—	30 RMB
SMS / email	low volume	150 RMB
LLM API (5 trial customers, “quality first”)	see §6.8	400 RMB
Monitoring	Grafana Cloud	0 RMB

Item	free Spec	Monthly
Total		**~3,180 RMB/month**

Supports ~5–10 trial/early customers.

Tier C full bill

Item	Spec	Monthly
Control node	Alibaba 4C8G	400 RMB
Worker nodes ×2	Alibaba 8C16G ×2	1,600 RMB
PostgreSQL	2C4G + 200GB	500 RMB
Redis	1C2G HA	200 RMB
OSS + CDN	—	300 RMB
Domain + SSL	—	30 RMB
SMS / email	—	200 RMB
LLM API (20 active, quality first)	see §6.8	1,500 RMB
Monitoring	—	0 RMB
Total		**~4,730 RMB/month**

Supports ~20–30 customers.

Revenue check (3 paying customers, tier A):

- Revenue: `2,999 × 2 + 999 = 6,997 RMB`
- Spend: ~3,200 RMB
- **Net gross profit ~~3,800 RMB~~ (54%)**—profitable from the start

By end of M3 with 15–20 paying customers (upgrade to B or C):

- Revenue ~40k–60k RMB/month
- Spend ~4,000–4,800 RMB
- **Net margin ~89–92%**

6.4 Growth phase bill (M3–M6, 30–150 customers)

Item	Spec	Monthly
K3s cluster	1 control + 2 workers (8C16G → 16C32G)	3,200 RMB
Serverless overflow	Alibaba SAE per-second, average	800 RMB
PostgreSQL	4C8G + 300GB + read replica	1,200 RMB
Redis	HA + sentinel	500 RMB
OSS + CDN (2× traffic)	—	1,000 RMB
LLM API (80 customers, quality first §6.8)	Claude + GPT-4o main	8,500 RMB
SMS / email	—	500 RMB
Proxies (GEO only; no auto-post)	—	500 RMB
Monitoring (self-hosted Grafana + Loki small VPS)	—	200 RMB
Total		**~16,400 RMB/month**

Supports ~80–150 customers.

Revenue check (80 customers):

- ARPU ~1,800 RMB (mostly entry, some pro, few flagship)
- Revenue `80 × 1,800 = 144,000 RMB`
- Spend 16,400 RMB
- **Net gross ~~128k RMB (~~89%)**

> ⚠ vs. "cheapest models" stack (~9,400 spend, 93% margin): quality-first costs ~7k more/month and ~89% margin—but **better outcomes → retention and upsell; higher LTV**. SaaS should not trade long-term churn for short-term margin.

6.5 Scale phase bill (M6–M12, 150–500 customers)

Item	Spec	Monthly
Managed K8s (ACK Pro)	cluster fee	600 RMB
Worker nodes (5 fixed + serverless mix)	—	8,000 RMB
PostgreSQL	8C16G primary + read replicas	3,000 RMB
Redis cluster	4 nodes	2,000 RMB
OSS + CDN + bandwidth	—	3,000 RMB
LLM API (500 customers, quality first §6.8)	Claude + GPT-4o main, DeepSeek assist	58,000 RMB
SMS / email / misc	—	2,000 RMB
Proxy pool	—	1,500 RMB
Monitoring / backup / DR	—	1,000 RMB
Total		**~79,000 RMB/month**

Supports ~500 customers.

Revenue check (500 customers):

- ARPU ~2,000 RMB (mature upsell mix)
- Revenue `500 × 2,000 = 1,000,000 RMB`
- Spend 79,000 RMB
- **Net gross ~~921k RMB (~~92%)**

> ⚠ LLM is the largest line (~73%)—do not compromise customer outcomes. Further savings: **prompt optimization** and **small-model distillation**, not downgrading the main model.

6.6 Very large scale (M12+, 500–3000 customers)

Item	Monthly
Multi-AZ K8s (China primary + overseas nodes)	25k RMB
Sharded/partitioned DB + replicas	15k RMB
Expanded cache cluster	8k RMB
LLM API (3000 customers, quality first)	280k–350k RMB
Proxies + account pool	15k RMB
Storage + CDN + cross-border	20k RMB
Monitoring + ops + DR	10k RMB
Total	**~370k–440k RMB/month**

Revenue check (3000 customers):

- ARPU ~2,000 RMB
- Revenue `3000 × 2,000 = 6,000,000 RMB`
- **Net gross ~~5.56M–5.63M RMB (~~93%)**

> ⚠ At this scale LLM is still ~70% of cost. Start **proprietary small models**—distill vertical models on your data; cut high-frequency LLM cost 60–80%. One-time investment ~500k–1M RMB; payback ~3 months.

6.7 Per-customer infra cost (the key numbers)

Larger scale → lower per-customer cost—SaaS leverage (quality-first):

Customers	Monthly infra	LLM share	Per-customer infra	Margin (ARPU 2000)
10	3,180 RMB	13%	**318 RMB/customer**	84%
30	6,200 RMB	35%	**207 RMB/customer**	90%
100	19,000 RMB	55%	**190 RMB/customer**	91%
500	79,000 RMB	73%	**158 RMB/customer**	92%
3000	400,000 RMB	77%	**133 RMB/customer**	93%

Takeaways:

- **~30 customers is the inflection**—margin stabilizes ~90% after
- **Cost does not keep dropping linearly**—LLM grows with usage
- **LLM share rises** (13% → 77%)—the price of "quality first"
- **Real savings**: distillation + prompt work, not cheaper flagship models

6.8 LLM API cost breakdown (quality-first strategy)

Principle: tier by scenario, not by cheapest model

LLM directly drives customer outcomes. **Customers paying 2,999 are not buying "AI that writes"—they buy traffic and conversion.** Saving LLM money and losing one customer wipes out a year of "savings."

Correct approach:

- **High-value** (customer-visible) → best models
- **Low-value** (internal, analytics) → cheaper models
- **Never save money by sacrificing outcomes**—first rule of SaaS

Model unit prices (2025–2026)

Model	Input / 1K tokens	Output / 1K tokens	Tier
DeepSeek V3 (cache hit)	0.0005	—	Ultra cheap (system prompts)
DeepSeek V3	0.002	0.008	**Cheap**
Qwen Max	0.004	0.012	Cheap
GPT-4o mini	~0.003	~0.01	Cheap, decent
GPT-4o	~0.018	~0.072	**Mid-high**
Claude 3.7 Sonnet	~0.022	~0.108	**Mid-high; best writing**
Claude 3.7 Opus	~0.108	~0.54	Top (flagship core)
o1 / o3	Higher	Higher	Reasoning flagship

Scenario-based routing

Scenario	Primary model	Why	Cut cost?
Content (published)	**Claude 3.7 Sonnet** + GPT-4o	Brand quality	✗ No
GEO prompt design	Claude 3.7 Sonnet	Core moat	✗ No

Scenario	Primary model	Why	Cut cost?
AI CS (flagship)	GPT-4o	Customer-visible	▯ No
AI CS (entry)	GPT-4o mini	Good enough	▯ Some
Flagship deep insights	**Claude 3.7 Opus**	Flagship differentiator	▯ No (use best)
RAG (internal)	DeepSeek V3	Not customer-facing	▯ Yes
Keyword mining	DeepSeek V3	Structured	▯ Yes
Ranking / competitor monitoring	DeepSeek V3	Analytics	▯ Yes
Content QA (second pass)	Claude 3.7 Sonnet	QA affects outcomes	▯ No

Mix by plan

Plan	Model mix	LLM / customer / month
999 entry	60% GPT-4o mini + 30% DeepSeek + 10% Claude (QA)	**~40 RMB**
2,999 pro	40% Claude 3.7 Sonnet + 30% GPT-4o + 30% DeepSeek	**~130 RMB**
9,999 flagship	50% Claude Opus + 30% Claude Sonnet + 20% GPT-4o	**~450 RMB**
Custom	As needed; often Opus-heavy	500–1,500 RMB/month

LLM monthly cost by customer mix (quality-first)

Mix	Monthly LLM
5 early customers (all entry)	~200 RMB
20 customers (15 entry + 5 pro)	~1,250 RMB
50 customers (30 entry + 15 pro + 5 flagship)	~5,400 RMB
100 customers (50 entry + 40 pro + 10 flagship)	~11,700 RMB
300 customers	~35,000 RMB
500 customers	~58,000 RMB

Cost controls (without hurting quality)

"Compliant savings"—cut waste, not quality:

1. **Prompt caching**: system prompts at cache rates—DeepSeek cache ~1/4 of list price
2. **Output caps**: `max_tokens` per scenario—no rambling
3. **Tiered calls**: cheap draft → premium polish (~50% cheaper than one-shot premium)
4. **Batching**: multiple keywords per request when possible
5. **Local fallback**: tiny model for ultra-high-frequency tasks (e.g. keyword expansion)
6. **Smart routing**: complexity-based model choice (mini vs Opus)

These can save another 20–30% without visible quality loss.

6.8.1 Outcomes are more than models

LLM is one link. **Same Claude; different prompts and data = your moat.** Full stack:

Layer	What	Importance
1. Models	Right model per scenario	▯▯▯
2.	**Industry prompt libraries** (manufacturing vs cross-border,	

Prompts Layer	iterated) What	⬜⬜⬜⬜⬜ Moat Importance
3. RAG	Deep enterprise KB in context	⬜⬜⬜⬜⬜
4. QA	Generate → second AI review → regen if fail	⬜⬜⬜⬜
5. Human	Advisor review for flagship/custom	⬜⬜⬜

Differentiation is layers 2–3—but models are the floor: weak models cap prompt/RAG upside.

6.9 Proxies and account pools ("hidden cost" decisions)

Item	Monthly estimate
Residential proxies (anti-bot)	500–3,000 RMB
Datacenter proxies	200–1,000 RMB
Platform accounts (rented/farmed)	500–3,000 RMB
Captcha services	100–500 RMB

⚠ **MVP decision: no auto-posting**

- Ship "AI generate + one-click copy to platform"
- Account pool cost → ~0
- Avoid anti-bot / ban whack-a-mole
- Reduce legal/platform risk
- **Save ~1,500–7,500 RMB/month**

6.10 ROI comparison (year 1 overall · quality-first)

Month	Customers	Monthly infra	Monthly SaaS revenue	Cumulative infra	Cumulative SaaS revenue	Net cumulative
M2	5	3,200	10k	6,400	10k	+3,600
M4	15	5,500	30k	30,400	60k	+29,600
M6	40	10,500	80k	82,400	200k	+118k
M9	80	16,400	160k	165,600	560k	+394k
M12	150	28,000	300k	275,600	1.3M	**+1.02M RMB**

Year-1 infra cumulative: ~275.6k RMB
Year-1 SaaS revenue cumulative: ~1.3M RMB
Infra as % of SaaS revenue: ~21%

Conclusion: **Fully covered; year-1 net from SaaS ~1.02M RMB.**

This **excludes custom project revenue** (commercial plan §4: custom year-1 ~24–30M RMB; many bring their own servers/on-prem—little shared SaaS infra).

6.11 vs. commercial plan total budget

In `[引擎一+定制化-商业化落地方案.md](./引擎一+定制化-商业化落地方案.md)` §5.2, year-1 infra + AI API budget was **2.3M RMB**.

With this cloud-native + quality-first model:

Line	Year-1 actual
SaaS infra (incl. LLM)	276k RMB
Custom project infra (~half bring own servers)	~350k RMB
LLM API (custom, heavy premium use)	~800k RMB
Proxies + third-party	~200k RMB

Total	**¥1,626M RMB**

~670k RMB under prior estimate—cloud-native + smart routing.

Suggested use of savings:

- 300k → marketing / industry events (revenue)
- 200k → extra hire (e.g. +1 sales)
- 170k → reserve / LLM spike buffer

6.12 Why this design

Principle	Meaning	Logic
OSS stack	K3s/KEDA/PG free	No vendor license tax
Elastic pools	Base + elastic + serverless overflow	Sublinear cost growth
Quality first	Premium models for customer-visible work	Outcomes = SaaS lifeblood
Tiered models	Pay more where it matters	Spend where ROI is clear
Scale leverage	~30 customers → ~90% gross margin	SaaS economics

One line: **OSS saves "dead money," tiered models save "wasted money," quality-first does not save "live money."**

---

7. Scaling strategy: 10 to 1000 customers

Answer is not "add VPS"—it's "change resource model by phase."

7.1 Scaling mindset

Traditional (wrong):

```
More customers → add VPS → manual placement → repeat
```

Cost grows **linearly** with customers.

Cloud-native (right):

```
Base pool (fixed) + elastic pool (on demand) + serverless burst (per second)

More customers → deeper queues → KEDA adds pods
              → more pods → cluster autoscaler adds nodes
              → peak overflow → serverless
              → peak ends → scale back
```

Cost grows **sublinearly** (~3× cost for ~10× scale).

7.2 Five scaling phases

Phase A (0–30 customers) · MVP

Resource	Config
Base pool	3 VPS (1 control + 2 workers)
Elastic	Not needed
Serverless	Not needed
Database	Small managed PG

Cost: ~3,500 RMB/month · ~100 RMB/customer

Phase B (30–100) · First elasticity

Resource	Config
Base pool	K3s 4–6 nodes
Elastic	**KEDA on**, workers 2–20 from queue
Serverless	Occasional overflow (~500–1500 RMB/month)
Database	Primary/replica + cache

Cost: 8,000–12,000 RMB/month · ~80–120 RMB/customer

Must-do: enable KEDA by M3—or you're just moving boxes.

Phase C (100–300) · Mature cloud-native

Resource	Config
Base pool	K3s 6–10 nodes or move to ACK
Elastic	KEDA + **Temporal**
Serverless	Regular overflow (~30% of burst)
Database	Shard or partition
Observability	Self-hosted Grafana LGTM

Cost: 20k–40k RMB/month · ~70–120 RMB/customer

Phase D (300–1000) · Scale-out

Resource	Config
Base pool	Managed K8s (ACK Pro / TKE)
Elastic	Cluster Autoscaler for nodes
Serverless	50%+ of burst on serverless
Database	Primary/replica/read paths + sharding
Cache	Redis cluster

Cost: 60k–120k RMB/month · ~60–120 RMB/customer

Phase E (1000+, incl. overseas) · Multi-region

Resource	Config
China	ACK (Beijing + Shanghai dual AZ)
Overseas	AWS EKS (Singapore) / Tencent overseas
Data	Regional deployments for compliance
Global	Service mesh + active-active patterns

Cost: 200k–500k RMB/month · ~50–80 RMB/customer

7.3 Triggers (systems scale themselves)

Signal	Action
Queue depth > 10	KEDA +1 worker pod
Queue depth > 100	+10 workers
Worker CPU > 70% for 5 min	Node autoscaler +1 node
Tenant QPS spike	Rate-limit + alert humans
DB connections > 80%	Pool tuning / scale instance
Storage > 80%	Expand or archive

Rules live in K8s config—**no human in the loop**.

7.4 Why this beats “add VPS”

Dimension	Add VPS	Cloud-native elastic
Response	Manual, hours	Automatic, seconds
Utilization	30–50% idle	70–90%
Cost	Linear	Sublinear
Recovery	Manual failover	Reschedule automatically
Multi-region	Hard	Config-driven
Ops headcount	Higher	Lower (small team viable)

---

8. 1–2 month MVP sprint

Week 1 · Architecture lock + skeleton

Day	You	Technical co-founder
Mon	Workshop: walk this plan; produce kickoff memo	Same
Tue	ICP filing + payment merchant onboarding	New repo + CI/CD + single-node K3s
Wed	List 5 design-partner customers	PostgreSQL schema + RLS design
Thu	Customer conversations	Xia Dockerfile + local task POC
Fri	Demo Day: K3s + Xia container	Same

Week 2 · Control plane skeleton

Module	Deliverable
Tenant service (signup/login/RLS)	Signup, login, `tenant_id` everywhere
Frontend `app.pont-ai.com` shell	Signup, login, dashboard shell
Intake flow	4-step onboarding
Redis Streams queue	Enqueue/dequeue works

Module	Deliverable

Week 3 · Task orchestration

Module	Deliverable
BullMQ / Celery workers	3 containers running jobs in parallel
Task types	Keyword mining, content gen, performance fetch
Console task list	Visible progress
K3s worker pods	Manual `kubectl scale`

Week 4 · Subscription + payment

Module	Deliverable
Plan definitions	3 tiers; seed DB + admin edits
WeChat Pay	Initiate / pay / webhook
Alipay	Same
Trial	Auto 30 days
Invoices	Form + manual invoicing

Week 5 · Quotas + billing

Module	Deliverable
Quota service	Pre-check, debit, alerts
Overage	Deduct prepaid balance
Monthly statement	Cron on 1st
Console usage page	Used / remaining

Week 6 · Deep Xia integration

Module	Deliverable
Customer data → KB	pgvector chunk + embed
KB retrieval	Vector search
LLM	DeepSeek primary + Qwen backup
Real customer jobs	≥10 articles shipped

Week 7 · AI CS agent

Module	Deliverable
JS SDK	One-line embed
RAG chat	On KB

Module	Deliverable
Human handoff	Email + back office
Demo site	Realistic demo

Week 8 · Observability + UAT

Item	Status
Grafana dashboards	Business + technical
Alerts	SMS / DingTalk
E2E UAT	You run 3 full passes
Load test	50 concurrent signups, 100 concurrent tasks
Legal	ToS / privacy

Week 9 · KEDA + private beta

Item	Deliverable
KEDA (event-driven scale)	Deeper queue → more pods
Internal + 3 design partners	Feedback + bugfix
Seed round pitch	Start

Week 10 · GA

- `/product` pricing live on marketing site
- `app.pont-ai.com` open signup
- 3 design partners convert to paid
- **No new features this week**—stability only

End of 2 months

□ K3s + KEDA running
□ Xia event-driven
□ Multi-tenant E2E
□ 3–5 paying customers
□ Autoscale validated
□ Seed round first close

---

9. Connecting to custom delivery

9.1 Shared vs. separated

```
Shared:
  - Xia engine code
  - LLM prompt libraries
  - KB / vector retrieval
  - Content templates

Per customer:
  - Data (`tenant_id`)
  - Deploy instance (K8s Namespace / dedicated cluster)
  - Industry plugins (feature flags)
```

9.2 Three deployment modes (same codebase)

Mode	Customer	Implementation
Multi-tenant SaaS	Entry / pro	Shared K3s, RLS isolation
Dedicated Namespace	Custom ¥5M	Same cluster, own namespace + ResourceQuota
Dedicated cluster / on-prem	Overseas / strict compliance	Helm one-click customer env

Key: **Same repo**—only deployment config differs. That is the value of the deployment abstraction.

9.3 When custom requests arrive

1. Sales captures → review (you + technical co-founder)
2. Triage:

- **~80% general** → mainline, all customers benefit
- **~15% industry plugins** → optional modules
- **~5% bespoke** → customer branch, not merged to main

3. Co-founder: architecture + critical code only; day-to-day to engineers + ops

---

10. Risks and mitigations

10.1 Technical

Risk	Mitigation
K3s/K8s ops harder than expected	MVP on managed ACK standard tier
Temporal learning curve	Migrate at M3; MVP on Redis Streams
Xia behaves differently containerized	POC in W2; buffer validation time
Multi-tenant data leak	RLS dual review + pentest before GA
Queue backlog	KEDA scale + alerts
LLM rate limits	Multi-vendor failover
Proxies blocked	MVP: no auto-post
DB hot spots	Slow-query monitoring; read replicas by M3
Dapr/Wasm new-tech risk	Optional—not mandatory

10.2 Operations

Risk	Mitigation
Single VPS failure	K3s multi-node, reschedule pods
Data loss	Daily cross-region OSS backup + 30-day retention
Team illness	Runbooks + on-call rotation
Signup abuse	Activation gates + WAF + rules engine

10.3 Evolution

Risk	Mitigation
Runaway stack churn	Every phase needs a **rollback story**
Over-engineering	No full K8s/Temporal in MVP—data decides
Cloud lock-in	Dapr + abstraction for portability

---

11. Five actions this week

Day	Action	Owner
Today	Send doc to co-founder; read by Sunday	You
Mon	4h workshop → kickoff memo	Both
Tue	ICP + payment onboarding / local K3s + repo skeleton	You / co-founder
Wed	List 5 design partners / `tenants` + `subscriptions` + `quotas` schema	You / co-founder
Thu	Customer calls / single worker runs a job end-to-end	You / co-founder
Fri	Demo Day: K3s + Xia worker pod + signup flow	Both

---

12. Appendix

12.1 Key decisions at a glance

Topic	MVP	Long-term	Rationale
Orchestration	Single-node K3s	Managed K8s / serverless	Light start; smooth upgrade
Task engine	Redis Streams + BullMQ	Temporal	Swap when workflows mature
Autoscale	Manual	**KEDA**	Must enable W9
Deployment abstraction	—	Dapr (optional)	After M6 by scale
Isolation	RLS + Namespace	+ NetworkPolicy / Wasm	Strengthen over time
Frontend	Next.js 16	Same	Align with `website/`
Backend	Co-founder strength	Same	Velocity
LLM	DeepSeek + Qwen	+ international models	Cost first
Payments	WeChat + Alipay	+ Stripe overseas	Global later
Auto-post	**Not in MVP**	Revisit M6 by anti-bot maturity	Risk reduction
Observability	Grafana Cloud free	Self-hosted LGTM	Fast start
Multi-cloud	Alibaba	+ AWS/GCP	Follow overseas deals

12.2 Technical milestones

Milestone	When	Metric
T0	W1	K3s + repo
T1	W2	Multi-tenant signup/login
T2	W4	Payments + subscriptions
T3	W6	Xia ↔ SaaS integrated
T4	W8	E2E UAT pass
T5	**W10**	**GA + KEDA live**
T6	M4	Dedicated Namespace deploy
T7	M6	Helm on-prem one-click
T8	M9	Managed K8s + serverless overflow
T9	M12	Multi-region (China + Singapore)

12.3 Improving Xia quality in parallel

Once SaaS is live, Xia issues surface at scale—good:

1. W5–W6: After SaaS launch ~1–2 weeks, bucket feedback into Top 20 issues
2. M2–M3: Prioritize Xia core; weekly image releases (K8s rolling update)
3. M3+: Baseline metrics (content pass rate, rank lift, hit rate)
4. M6: "Xia v2" benefits custom customers too

Key: ship "good enough"; iterate on real usage—not wait for perfection.

12.4 Openclaw integration path

Time	State
Now	Openclaw on dedicated VPS for SEO
Before M2	Leave as-is
M3–M6	Gradually become a Xia worker type
M6+	Fully internal module; unified scheduler

---

13. Document map

Document	Role	Audience
`[公司介绍.md](./公司介绍.md)`	External brand	Customers, investors
`[引擎一-订阅与用量收入规划.md](./引擎一-订阅与用量收入规划.md)`	Product definition	Whole team
`[引擎一+定制化-商业化落地方案.md](./引擎一+定制化-商业化落地方案.md)`	Commercial strategy	You / co-founder / investors
This doc (v2.0)	**Technical execution map**	**Co-founder / engineers / ops**

This document is the single source of truth for technical execution.
Core principles: deployment abstraction + event-driven scaling + multi-tenant isolation + progressive evolution.

Every Friday Demo Day: check progress against this doc; if drift >1 week, run a retrospective.